

Dynamics and Stability of the Spring-Loaded Inverted Pendulum (SLIP) Model

Arael Anaya

1 Introduction and Relevance

The spring-loaded inverted pendulum (SLIP) is the reduced-order model behind most legged-robot dynamics [3, 5]: a point mass bouncing on a massless spring leg. Despite its simplicity, it captures the center-of-mass dynamics of running in both animals and machines remarkably well [1, 2]. That makes it a good subject for this course: it is simple enough to analyze fully with the tools covered here, but rich enough to produce genuinely interesting hybrid dynamics. It also builds directly on the elastic pendulum studied in class, since the stance phase of SLIP is an inverted spring pendulum.

What makes SLIP especially interesting from a mechanics standpoint is that it is a hybrid dynamical system: during stance the body behaves like a spring-loaded pendulum, while in flight it is a free projectile, with the two phases connected by touchdown and liftoff switching conditions. Rather than analyzing stability at a fixed equilibrium point, the more natural question for a hopping gait is whether the periodic motion itself is stable. The standard tool for that is the apex return map, a Poincaré map in which a steady hopping gait appears as a fixed point whose stability can be assessed numerically [1, 4].

Beyond being a convenient classroom system, SLIP-type models are used directly in the design and control of running robots and in the biomechanical study of animal and human locomotion, which is the practical motivation for this project: a model simple enough to analyze by hand but predictive enough to inform how a real legged system should be built or controlled.

Relation to Prior Work

This project sits within an established line of SLIP research, and each stage of the model draws on a different piece of that literature. Blickhan [1] introduced the spring-mass template and showed that it reproduces the ground-reaction-force and center-of-mass patterns of human and animal running; the stance-phase Lagrangian derived here reproduces that same reduced-order model, but from Lagrange's equations in polar coordinates rather than Blickhan's Newtonian force balance. McMahon and Cheng [2] used the same template to relate leg stiffness to running speed empirically; the parameter sweep over stiffness k and touchdown angle θ_{TD} planned for this project asks the same question, but numerically, from the model directly, rather than from experimental data. Full and Koditschek's templates-and-anchors framework [3] is the justification for treating SLIP as a legitimate object of study on its own, rather than only as a shortcut on the way to a fully articulated, multi-link leg model: a template is expected to capture the essential dynamics of the anchored system while discarding detail that does not matter for stability. Schwind and Koditschek [4] derived an approximate, closed-form stance map for SLIP using a perturbative solution; this project instead integrates the stance-phase equations numerically with an event-based solver, trading their analytical insight for an exact (if less interpretable) stance map, consistent with the computational tools emphasized in this course. Saranli, Buehler, and Koditschek [5] built RHex, a hexapod whose individual legs behave like a SLIP template during stance, which is the practical grounding for why this abstraction is worth studying: it is not just a toy system, it is the model actually used to understand a working robot. Finally, Geyer, Seyfarth, and Blickhan [6] extended a SLIP-type compliant-leg model to walking, not just running, by allowing double support; that extension is a natural direction for future work beyond the single-support hopping model considered here.

Expand this comparison if needed to more explicitly satisfy the “at least 5 scholarly articles” requirement with direct quantitative comparisons (e.g., comparing this project’s stiffness sweep results directly against McMahon & Cheng’s reported stiffness–speed trend, or against Schwind & Koditschek’s closed-form stance-map approximation) once the parameter sweep in Section 4 is complete.

2 Modeling of Dynamics

The equations of motion for both phases of the SLIP model are derived using **Lagrange’s equations of motion**, with generalized coordinates chosen separately for each phase: Cartesian coordinates (x, y) for the flight phase, and polar coordinates (r, θ) about the planted foot for the stance phase. The two phases are connected by touchdown and liftoff switching (transversality) conditions, so the overall model is a hybrid dynamical system integrated with event-based numerical methods. Figure 1 shows the geometry of both phases.

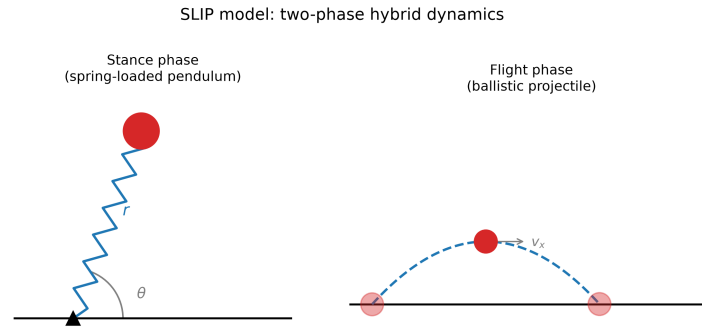


Figure 1: SLIP model geometry: stance (spring pendulum, r, θ) and flight (projectile).

2.1 Flight-Phase Equations of Motion

With basis $\hat{n}_1, \hat{n}_2$, $\vec{r}_m = x\hat{n}_1 + y\hat{n}_2$:

$$T = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2), \quad V = mgy, \quad L = T - V = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2) - mgy$$

Lagrange’s equations give

$$\ddot{x} = 0, \quad \ddot{y} = -g$$

Starting at the apex ($\dot{y} = 0$), with $x(0) = x_0$, $\dot{x}(0) = v_{x0}$:

$$x(t) = v_{x0}t + x_0, \quad y(t) = y_0 - \frac{g}{2}t^2$$

2.2 Touchdown: Detection and Transition Map

θ_{TD} is held fixed in flight, so the foot trails the body at

$$\vec{r}_{foot}(t) = (x(t) - r_0 \cos \theta_{TD})\hat{n}_1 + (y(t) - r_0 \sin \theta_{TD})\hat{n}_2$$

Touchdown is $y_{foot} = 0$, i.e. $y_{TD} = r_0 \sin \theta_{TD}$. Solving $y(t) = y_0 - \frac{g}{2}t^2$:

$$t_{TD} = \sqrt{\frac{2(y_0 - r_0 \sin \theta_{TD})}{g}}, \quad \dot{y}_{TD} = -\sqrt{2g(y_0 - r_0 \sin \theta_{TD})}, \quad x_{TD} = v_{x0}t_{TD}, \quad \dot{x}_{TD} = v_{x0}$$

Stance uses polar coordinates about the planted foot, so this Cartesian state must convert. With foot at $\vec{r}_F = (x_{TD} + r_0 \cos \theta_{TD})\hat{n}_1$ and leg vector $\vec{\rho} = \vec{r}_m - \vec{r}_F = -r_0 \cos \theta_{TD} \hat{n}_1 + y_{TD} \hat{n}_2$:

$$r = |\vec{\rho}| = \sqrt{r_0^2 \cos^2 \theta_{TD} + y_{TD}^2} = r_0, \quad \theta = \text{atan2}(y_{TD}, -r_0 \cos \theta_{TD})$$

Projecting $\vec{v} = \dot{x}_{TD} \hat{n}_1 + \dot{y}_{TD} \hat{n}_2$ onto $\hat{e}_r = \cos \theta \hat{n}_1 + \sin \theta \hat{n}_2$, $\hat{e}_\theta = -\sin \theta \hat{n}_1 + \cos \theta \hat{n}_2$:

$$\dot{r} = \vec{v} \cdot \hat{e}_r = \dot{x}_{TD} \cos \theta + \dot{y}_{TD} \sin \theta, \quad \dot{\theta} = \frac{\vec{v} \cdot \hat{e}_\theta}{r_0} = \frac{-\dot{x}_{TD} \sin \theta + \dot{y}_{TD} \cos \theta}{r_0}$$

implemented as `touchdownState()`.

2.3 Stance-Phase Equations of Motion

In stance the leg pivots about the fixed foot: $\vec{r}_m = r\hat{e}_r$, $\vec{v}_m = \dot{r}\hat{e}_r + r\dot{\theta}\hat{e}_\theta$:

$$T = \frac{1}{2}m\dot{r}^2 + \frac{1}{2}mr^2\dot{\theta}^2, \quad V = mgr \sin \theta + \frac{1}{2}k(r - r_0)^2, \quad L = T - V$$

Lagrange's equations:

$$m\ddot{r} - mr\dot{\theta}^2 + mg \sin \theta + k(r - r_0) = 0, \quad mr^2\ddot{\theta} + 2mr\dot{r}\dot{\theta} + mgr \cos \theta = 0$$

solved for the highest derivatives:

$$\ddot{r} = r\dot{\theta}^2 - g \sin \theta - \frac{k}{m}(r - r_0), \quad \ddot{\theta} = -\frac{2}{r}\dot{r}\dot{\theta} - \frac{g}{r} \cos \theta$$

implemented as `stance_RHS()`.

2.4 Conserved Energy in Stance

$\vec{r}_m = r\hat{e}_r$ is scleronomic and T is homogeneous quadratic in $(\dot{r}, \dot{\theta})$, so $H = T + V$; with no explicit t -dependence, energy is conserved through stance:

$$E = \frac{1}{2}m(\dot{r}^2 + r^2\dot{\theta}^2) + mgr \sin \theta + \frac{1}{2}k(r - r_0)^2 = \text{const.}$$

Neither coordinate is cyclic, so both equations must be integrated together; this conserved quantity is used below as a check on the numerical integrator.

2.5 Liftoff Map and the Apex Return Map

Liftoff occurs when the leg returns to its natural length with positive rate, $r = r_0$, $\dot{r} > 0$, at which point the stance-phase state $(r_0, \theta_{LO}, \dot{r}_{LO}, \dot{\theta}_{LO})$ must be converted back into Cartesian flight variables to begin

the next flight phase. Chaining flight $\rightarrow$ touchdown $\rightarrow$ stance $\rightarrow$ liftoff $\rightarrow$ flight once defines a map from one apex state to the next, $P : (y_n, v_{x,n}) \mapsto (y_{n+1}, v_{x,n+1})$, whose fixed points correspond to steady periodic hopping and whose local stability is governed by the eigenvalues of the linearized (Jacobian) map about that fixed point.

Derive the liftoff analogue of the touchdown map above (stance polar state $\rightarrow$ Cartesian $\dot{x}_{LO}, \dot{y}_{LO}$, then propagate to the next apex to get $y_{n+1}, v_{x,n+1}$), define the apex return map P explicitly as a function of (k, θ_{TD}) , and state the fixed point condition $P(y^*, v_x^*) = (y^*, v_x^*)$ used for the stability analysis in Section 4.

3 Discussion of Assumptions

Several modeling choices were made to keep the SLIP model tractable with the tools of this course, each with a corresponding limitation:

- **Planar motion.** The model is restricted to the sagittal plane. Real running involves some lateral and rotational motion of the trunk and legs, which this model cannot capture.
- **Point mass, massless leg.** All body mass is lumped at a single point, and the leg itself carries no mass or rotational inertia. This isolates the center-of-mass dynamics that SLIP is meant to describe, but discards any effect of leg-swing inertia or trunk pitch, which the templates-and-anchors view [3] explicitly treats as detail that a template is allowed to omit.
- **Linear (Hookean) leg spring.** The stance leg is modeled as a single ideal linear spring, $F = -k(r - r_0)$, with constant stiffness k . Biological and robotic legs have leg stiffness that is nonlinear and can be actively modulated by the neuromuscular system or a controller; treating k as a fixed parameter to be swept is a simplification, not a claim that real legs behave this way.
- **Fixed touchdown angle held through flight.** θ_{TD} is assumed constant for the entire flight phase, i.e. there is no swing-leg retraction or angle adjustment in response to a disturbance before the foot lands. This is the standard SLIP assumption, but it means the model's only "control input" between hops is the touchdown angle chosen before each flight, not a continuous correction during flight.
- **Instantaneous, rigid, non-slipping foot placement.** Touchdown and liftoff are treated as instantaneous events with no impact losses, and the foot is assumed to stick to the ground (no slipping, no ground compliance) for the duration of stance at a fixed height $y = 0$. Real contact involves finite compliance and a friction cone that could, in principle, be violated at high touchdown angles or high speeds.
- **No stance-phase dissipation, no aerodynamic drag.** The stance-phase Lagrangian has no damping term, so mechanical energy is exactly conserved within a single stance phase (verified numerically in Section 4); flight is likewise treated as drag-free projectile motion. This was a deliberate choice: it isolates the elastic dynamics of interest and gives a clean, near-machine-precision check on the numerical integrator, at the cost of neglecting the fact that any real leg or running surface dissipates some energy each stride.
- **Equal rest length at touchdown and liftoff.** The leg is assumed to return to the same natural length r_0 between hops, with no active leg extension/retraction actuation changing that rest length from one stance phase to the next.

If time allows, discuss which of the assumptions above are most consequential for the stability results in Section 4 (e.g., does relaxing the zero-damping assumption change which (k, θ_{TD}) combinations are stable?), and note anywhere the numerical results below reveal a limitation of the model (such as touchdown angles/speeds that would violate a real friction cone).

4 Computational Results

4.1 Single-Stance Validation

The model is implemented in Python. `main.py` maps an apex state to the stance-phase initial condition via `touchdownState()`, then integrates `stance_RHS()` with `solve_ivp` (`rtol` = 10^{-10} , `atol` = 10^{-12}) to the liftoff event $r = r_0$, $\dot{r} > 0$, checking the conserved energy from Section 2.4 at every step. Using the parameters below, this validates the stance-phase model end to end:

m	r_0	k	g	θ_{TD}	y_0	v_{x0}
80 kg	1 m	20000 N/m	9.81 m/s ²	68°	1.2 m	3.0 m/s

	r [m]	θ [deg]	$\dot{r}$ [m/s]	$\dot{\theta}$ [rad/s]
Touchdown	1.0000	112.00	−3.2689	−1.9149
Liftoff	1.0000	83.48	3.1937	−1.6886

Stance lasts 0.2094 s; the leg compresses to 0.7740 m (22.6% of r_0); relative energy drift is 6.46×10^{-13} , i.e. machine precision, confirming that the integrator and the equations of motion in Section 2.3 are consistent. Figure 2 plots leg length, angle, CoM trajectory, and energy drift for this run.

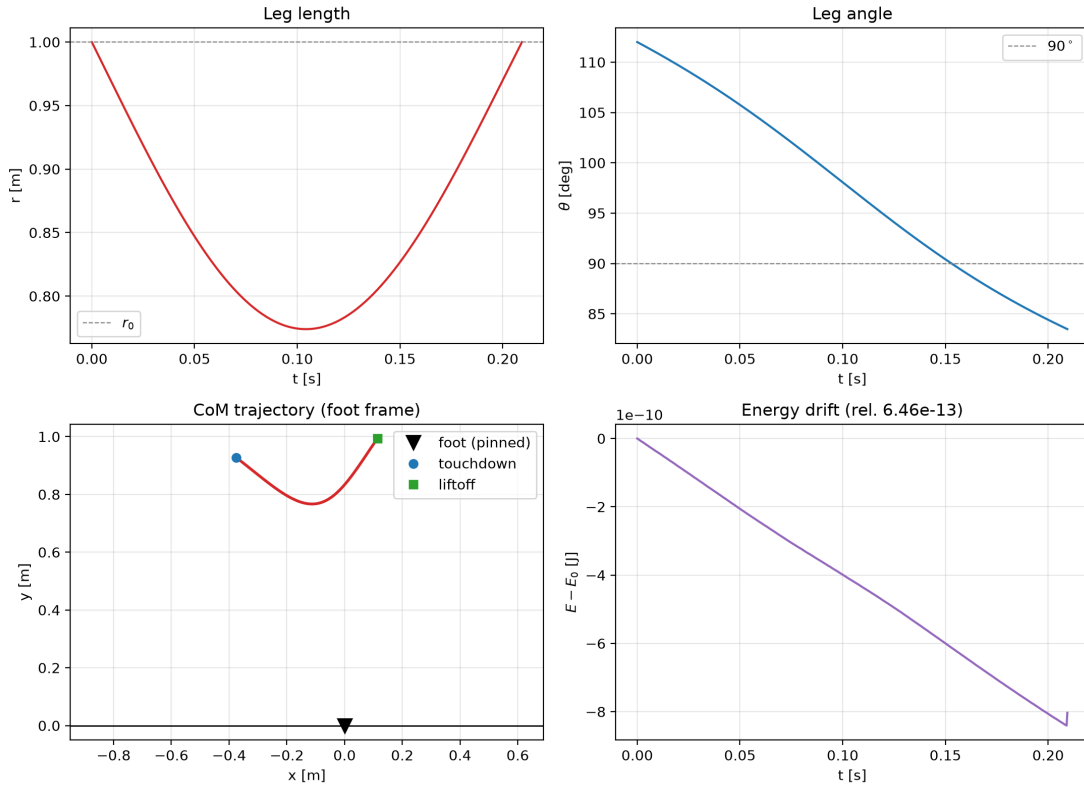


Figure 2: Stance simulation: $r(t)$, $\theta(t)$ (top), CoM trajectory (bottom left), energy drift (bottom right).

4.2 Apex Return Map, Phase Space, and Parameter Sweep

Once Section 2.5's liftoff/return map is implemented, populate this subsection with:

- A phase-space plot (e.g. $\dot{\theta}$ vs. θ , or successive apex heights y_{n+1} vs. y_n) showing convergence to a fixed point of the apex return map for at least one stable (k, θ_{TD}) combination.
- A sweep over spring stiffness k and touchdown angle θ_{TD} , reporting which combinations produce a stable steady hopping gait (e.g. eigenvalues of the linearized return map inside the unit circle) and which do not.
- A short interpretation of the sweep: does stability match the qualitative stiffness–speed trend reported by McMahon and Cheng [2]? Are there touchdown angles that fail even though the single-stance dynamics are otherwise well behaved?

5 Lessons Learned

Address all three of the following once the project is complete:

1. **Benefits and limitations of the chosen approach.** What did the Lagrangian/generalized-coordinates + event-based hybrid-integration approach make easy, and where did it break down or require extra care (e.g. the touchdown/liftoff coordinate conversions, or the accuracy demands of chaining many hops through the return map)?
2. **What was learned by modeling this system.** E.g. anything surprising about the stance dynamics, the sensitivity of the return map to (k, θ_{TD}) , or the practical challenges of event-based integration that weren't obvious before doing this project.
3. **Which course learning outcomes this project demonstrates.** Connect explicitly back to specific MEGN505 learning outcomes (Lagrangian mechanics, generalized coordinates, constraints, stability of periodic motion via Poincaré maps, numerical simulation of dynamical systems, etc.).

References

- [1] R. Blickhan, “The spring-mass model for running and hopping,” *Journal of Biomechanics*, vol. 22, no. 11–12, pp. 1217–1227, 1989.
- [2] T. A. McMahon and G. C. Cheng, “The mechanics of running: how does stiffness couple with speed?,” *Journal of Biomechanics*, vol. 23, suppl. 1, pp. 65–78, 1990.
- [3] R. J. Full and D. E. Koditschek, “Templates and anchors: neuromechanical hypotheses of legged locomotion on land,” *Journal of Experimental Biology*, vol. 202, no. 23, pp. 3325–3332, 1999.
- [4] W. J. Schwind and D. E. Koditschek, “Approximating the stance map of a 2-DOF monopod runner,” *Journal of Nonlinear Science*, vol. 10, pp. 533–568, 2000.
- [5] U. Saranli, M. Buehler, and D. E. Koditschek, “RHex: a simple and highly mobile hexapod robot,” *International Journal of Robotics Research*, vol. 20, no. 7, pp. 616–631, 2001.
- [6] H. Geyer, A. Seyfarth, and R. Blickhan, “Compliant leg behaviour explains basic dynamics of walking and running,” *Proceedings of the Royal Society B*, vol. 273, no. 1603, pp. 2861–2867, 2006.

Appendix: Code

`main.py` – touchdown map, stance-phase RHS, and the single-stance simulation and validation plots discussed in Section 4.1.

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 # Paramerts
5 m = 80
6 r_o = 1
7 k = 20000
8 g = 9.81
9 theta_TD = np.radians(68)
10 y_o = 1.2
11 v_o = 3.0
12
13 params = {'m' : m , 'k' : k , 'g' : g, 'r_o' : r_o}
14
15
16 def touchdownState(y_o , v_o, theta_TD, r_o, g):
17     r = r_o
18     theta = np.arctan2(r_o * np.sin(theta_TD) , -r_o * np.cos(theta_TD))
19
20     x_TD_dot = v_o
21     y_TD_dot = - (2*g*(y_o - r_o * np.sin(theta_TD))**.5
22
23
24     r_dot = x_TD_dot * np.cos(theta) + y_TD_dot * np.sin(theta)
25
26     theta_dot = (-x_TD_dot * np.sin(theta) + y_TD_dot * np.cos(theta)) / r_o
27
28
29
30     return [r , theta, r_dot , theta_dot]
31
32 def liftoff(t, s, params):
33     return params['k'] * (s[0] - params['r_o'])
34 liftoff.terminal = True
35 liftoff.direction = 1

```

```

36
37
38 def stance_RHS(t , s, params):
39     r_dot = s[2]
40     theta_dot = s[3]
41
42
43     r_ddot = s[0] * theta_dot**2 - params['g'] * np.sin(s[1]) - (params['k'] /
44         params['m']) * (s[0] - params['r_o'])
45     theta_ddot = -(2/s[0]) * r_dot * theta_dot - (params['g'] / s[0]) * np.cos
46         (s[1])
47
48     return [r_dot , theta_dot, r_ddot, theta_ddot]
49
50
51 def energy(s, params):
52     E = (1/2) * params['m'] * (s[2]**2 + s[0]**2 * s[3]**2) + params['m'] *
53         params['g'] * s[0] * np.sin(s[1]) + (1/2) * params['k'] * (s[0] -
54         params['r_o'])**2
55
56     return E
57
58 s0 = touchdownState(y_o , v_o, theta_TD, r_o, g)
59 t_span = (0.0,1.0)
60
61 sol = solve_ivp(stance_RHS, t_span, s0, args=(params,),
62     events=liftoff, rtol=1e-10, atol=1e-12,
63     dense_output=True, max_step=1e-3)
64
65 E = np.array([energy(sol.y[:, i], params) for i in range(sol.y.shape[1])])
66 drift = np.max(np.abs(E - E[0])) / np.abs(E[0])
67
68 print("status:", sol.status)
69 print("liftoff time:", sol.t_events[0])
70 print("drift:", drift)
71
72 import matplotlib.pyplot as plt
73
74 # ----- diagnostics -----
75 r_TD, th_TD, rdot_TD, thdot_TD = s0
76 s_L0 = sol.y_events[0][0]
77 t_L0 = sol.t_events[0][0]
78
79 print("--- touchdown state ---")
80 print(f" r          = {r_TD:.6f} m")
81 print(f" theta      = {np.degrees(th_TD):.4f} deg")
82 print(f" r_dot      = {rdot_TD:.6f} m/s")
83 print(f" th_dot     = {thdot_TD:.6f} rad/s")
84
85 print("--- liftoff state ---")
86 print(f" status      = {sol.status}")
87 print(f" stance duration= {t_L0:.6f} s")
88 print(f" r          = {s_L0[0]:.6f} m")
89 print(f" theta      = {np.degrees(s_L0[1]):.4f} deg")
90 print(f" r_dot      = {s_L0[2]:.6f} m/s")
91 print(f" th_dot     = {s_L0[3]:.6f} rad/s")

```

```

88 print("--- validation ---")
89 print(f"    min leg length = {np.min(sol.y[0]):.6f} m")
90 print(f"    max compression= {params['r_o'] - np.min(sol.y[0]):.6f} m")
91 print(f"    E_0                = {E[0]:.6f} J")
92 print(f"    relative drift = {drift:.3e}")
93
94 # ----- figures -----
95 t = sol.t
96 r = sol.y[0]
97 th = sol.y[1]
98 x_com = r * np.cos(th)
99 y_com = r * np.sin(th)
100
101 fig, ax = plt.subplots(2, 2, figsize=(11, 8))
102
103 ax[0, 0].plot(t, r, color='C3')
104 ax[0, 0].axhline(params['r_o'], ls='--', lw=0.8, color='gray', label='$r_o$')
105 ax[0, 0].set_xlabel('t [s]')
106 ax[0, 0].set_ylabel('r [m]')
107 ax[0, 0].set_title('Leg length')
108 ax[0, 0].legend()
109
110 ax[0, 1].plot(t, np.degrees(th), color='C0')
111 ax[0, 1].axhline(90, ls='--', lw=0.8, color='gray', label='$90^\circ$')
112 ax[0, 1].set_xlabel('t [s]')
113 ax[0, 1].set_ylabel(r'$\theta$ [deg]')
114 ax[0, 1].set_title('Leg angle')
115 ax[0, 1].legend()
116
117 ax[1, 0].plot(x_com, y_com, color='C3', lw=2)
118 ax[1, 0].plot(0, 0, 'kv', ms=10, label='foot (pinned)')
119 ax[1, 0].plot(x_com[0], y_com[0], 'o', color='C0', label='touchdown')
120 ax[1, 0].plot(x_com[-1], y_com[-1], 's', color='C2', label='liftoff')
121 ax[1, 0].axhline(0, color='k', lw=1)
122 ax[1, 0].set_xlabel('x [m]')
123 ax[1, 0].set_ylabel('y [m]')
124 ax[1, 0].set_title('CoM trajectory (foot frame)')
125 ax[1, 0].axis('equal')
126 ax[1, 0].legend()
127
128 ax[1, 1].plot(t, E - E[0], color='C4')
129 ax[1, 1].set_xlabel('t [s]')
130 ax[1, 1].set_ylabel('$E - E_0$ [J]')
131 ax[1, 1].set_title(f'Energy drift (rel. {drift:.2e})')
132 ax[1, 1].ticklabel_format(axis='y', style='sci', scilimits=(0, 0))
133
134 for a in ax.flat:
135     a.grid(alpha=0.3)
136 fig.tight_layout()
137 fig.savefig('stance_validation.png', dpi=200)
138 plt.show()

```

Once written, add the liftoff-map and apex-return-map / parameter sweep script here (e.g. `return_map.py`) with a matching `lstinputlisting` entry, alongside a short note describing what it computes.